

smallgameagent 实验报告

基于 LLM/VLM 与规则的小游戏 Agent

smallgameagent 项目组

2026 年 7 月 17 日

目录

1	smallgameagent 实验报告 (终稿)	2
1.1	0. 一页结论	2
1.2	1. 环境打通 (新信息)	3
1.3	2. 空间一致性 (问题)	3
1.4	3. 策略优化 (问题)	4
1.5	4. 效率 (问题)	5
1.6	5. 时间一致性 (问题)	5
1.7	6. 本地 VLM 画像 (P5)	5
1.7.1	6.1 文本生成基准(Q4_K_M,-ngl 99,prompt="1, 2, 3,",max_tokens=60)	5
1.7.2	6.2 视觉 struct 离线评测	5
1.7.3	6.3 结论	6
1.8	7. 训练准备 (P7, 等 ssh5090)	6
1.9	8. verifiers 风格环境 (P6)	7
1.10	9. 给同学四个问题的直接回答	7
1.11	10. 后续建议 (优先级序)	7

1 smallgameagent 实验报告 (终稿)

> 2026-07-17。配套:\texttt{EXPERIMENT_}_PLAN.md}(方案)、\texttt{EXPERIMENTV}(过程数据)。

> 本文面向合作同学与老师，按四个问题组织结论，附三条决策路线的效率-质量画像与训练准备状态。

1.1 0. 一页结论

- \textbf{空间一致性}：根因不是「记不住场景」而是\textbf{交互方式随场景切换失效未被检测}。实证：

00461 场景切换后 \texttt{LosePanel} 激活阻断摇杆输入，规则 agent 继续发移动指令瘫痪 175+ 步。

解法是\textbf{版本化世界模型 + 失效检测与恢复} (stale 标记、失败面板自动点击)，A/B 显示活动率 0.54→0.92、墙钟 263→154s (−42%)。

- \textbf{时间一致性}：交付\textbf{阶段契约机制} (三层时间对齐 + settle 复查 + 受保护前缀 hash +

失败分级 rollback/compensation/stop+replan)，29 单测含真实温度案例复刻 (131/190 步偏差)。

- \textbf{策略优化}：kimi-k2.7-code 的经济决策实验显示\textbf{解析与 token 预算才是第一瓶颈}

(4K max_tokens 下 58% 调用截断无答案)；修正后朴素与引导式在饱和场景打平 (0.93)，决定性场景 (batch 占优 3-4×) 结果见 §3。

- \textbf{效率}：动态探针预算模块 (五触发器、L0/L1/L2 分层、L2 预算上限 20%) + 动态日志分级，

27 单测，模拟装饰性场景节省观测成本 ~82%。

- \textbf{VLM 本地推理}：Intel 核显 (WSL2 Mesa Dozen/Vulkan) 文本基准实测：

\textbf{Qwen3.5-4B 2.75 tok/s、Qwen3.5-9B 0.72 tok/s、gemma-4-E4B 0.67 tok/s} (Q4_K_M, -ngl 99)。

视觉编码必须 `\{\}\texttt{-no-mmmproj-offload}` 回 CPU，否则挂死或输出空白；4B 视觉 struct 4 帧评测

`\{\}\textbf{解析率 0/4}`（超时/连接抖动），平均 534 s/帧，`\{\}\textbf{当前仅适合离线标注，不适合在线逐步控制}`。

- `\{\}\textbf{训练}`：processed-runs 已转为 7 任务 15,083 样本（14 单测 + 加载验证），

训练脚本就绪，等 ssh5090 可访问后开 QLoRA。

1.2 1. 环境打通（新信息）

项	结论
云端 LLM	<code>\{\}\texttt{https://opencode.ai/zen/go/v1}</code> ，kimi-k2.7-code 文本决策 2.4–6.3s/次（旧 deepseek
本地 VLM	LM Studio 捆绑 llama.cpp Vulkan 后端（WSL2 Mesa Dozen→Intel Graphics）；lms CLI 因版本
游戏基线	rule 模式 SSD_00461 塔防 300 步基线 composite 0.425（verifiers 风格 rubric）

1.3 2. 空间一致性（问题）

`\{\}\textbf{实证根因}`（三级递进，全部有 run 数据）：

1. 初判「场景切换后撞新障碍卡死」→ 障碍学习 + 势场避让（P1）上线后仍失败；
2. 再判「英雄死亡」→ 诊断跑血量全程 100，排除；
3. `\{\}\textbf{定案}`：failCount 翻转 → LosePanel 激活 → 输入被面板阻断，agent 不点继续 → 永久瘫痪。

这正是同学说的「交互方式改变」类失效（摇杆交互被模态面板替换）。

`\{\}\textbf{修复}`：

- `\{\}\texttt{VersionedWorldModel}` (`\{\}\texttt{src/agent/world\_model.py}`): entity_version/scene_epoch/capability_epoch,

观测写入比对、stale 标记、局部重规划，12 单测（含 autoFishing 12 次翻转复刻）。

- 探针新增 `\{\}\texttt{findPanelButtons}`：按`\{\}\textbf{节点名}`匹配按钮（生产构建组件名被 minify 成单字母，

按类名匹配必然落空），返回设计分辨率坐标；Python 侧 retry 优选、避开 download 广告陷阱，

设计坐标 → CSS 像素映射（720×1560 左下原点 → 375×812 顶左）。7 单测。

- RuleEngine 障碍学习（势场避让 + 定向逃逸，15 单测）——对几何卡死有效，但非本游戏 binding。

$\backslash\{\}\text{textbf{A/B}}$ (00461, 300 步, rubric) }:

run	composite	activity	progress	stall 步	墙价
基线	0.425	0.535	0.667	139	262
+ 世界模型/障碍	0.351	0.408	0.583	177	320
$\backslash\{\}\text{textbf{+ 失败面板处理}}$	$\backslash\{\}\text{textbf{0.473}}$	$\backslash\{\}\text{textbf{0.920}}$	$\backslash\{\}\text{textbf{0.750}}$	$\backslash\{\}\text{textbf{24}}$	$\backslash\{\}$

1.4 3. 策略优化（问题）

实验：12 经济场景（gold/双升级项/收入/预算），最优解由穷举模拟器给出，kimi-k2.7-code

朴素 prompt vs 机制引导 prompt（先抽象经济循环 + 往返成本再决策）。

- v1 (max_tokens 4096): $\backslash\{\}\text{textbf{14/24}}$ 次调用无答案——思考型模型把 token 预算耗尽于推理，

JSON 被截断。教训：思考模型必须留足输出预算（16K）并约束「末行只放 JSON」。

- v2 (16K tokens): 解析失败归零；朴素 vs 引导 0.929 vs 0.932（场景饱和，11/12 greedy 最优，无法区分）。
- v3 (8 个 batch 占优 3-4× + 4 对照，模拟器穷举真值):

$\backslash\{\}\text{textbf{最优命中率朴素 41.7\{\}\%}}$ vs 引导 $66.7\{\}\%$ ；按得分比率（所选策略得分/最优得分）:

* 朴素 batch=0.711 / control=1.000 (overall 0.808); 引导 batch=1.000 / control=0.958 (overall 0.986) **。

结论：机制引导把 batch 经济决策从 71% 提升到 100% 最优值，代价仅 greedy 场景 -4%——

定量证实了「加以引导后探索最优解能力强」的判断。引导的关键是 prompt 里显式给出

「经济循环 + 往返/机会成本」的抽象，而不是让模型自己发现。

1.5 4. 效率 (问题)

- `\{\}\texttt{src/agent/probe\{\}\_budget.py}`: L0 状态 (73ms) /L1 组件 (150ms) /L2 截图 (400ms) 三级,

五触发器(phase_change/low_confidence/action_no_effect/collision_hint/semantic_flip), 冷却防抖 + L2 窗口占比上限 (默认 20%) + 高优先挤占; AdaptiveLogger 环形内存 DEBUG + 触发窗口落盘。27 单测; 50 步无变化场景模拟节省 81.75%。

- 失败面板自动恢复即「策略级回退」的首个实例 (行为级: 障碍逃逸; 状态级: stale 重规划;

策略级: 面板 dismiss)。

1.6 5. 时间一致性 (问题)

`\{\}\texttt{src/agent/phase\{\}\_contract.py}` (29 单测): TimestampedValue 三层时间戳 (event/observed/settled)、跨字段一致性校验器 (复刻 190 步 131 违例)、PhaseGate 状态机 (settle 复查防完成假阳性、守卫违反 VIOLATED、超时 TIMEOUT)、受保护前缀 hash (篡改即 PrefixViolation)、失败按副作用分级。与游戏 loop 的集成 (契约守卫替换硬编码 fail 检测) 是下一步。

1.7 6. 本地 VLM 画像 (P5)

1.7.1 6.1 文本生成基准(Q4_K_M,-ngl 99,prompt="1, 2, 3",max_tokens=60)

模型	server tok/s	wall 60 tokens	备注
Qwen3.5-4B	$\backslash\{\}\texttt{2.75}$	24.6 s	可用
Qwen3.5-9B	0.72	90.3 s	慢但可跑
gemma-4-E4B-it	0.67	37.5 s	输出格式偏闲聊, 需强约束

1.7.2 6.2 视觉 struct 离线评测

- 4 帧真实截图(`\{\}\texttt{SSD\{\}\_00496P01}` step 2/11,`\{\}\texttt{SSD\{\}\_00219P01}` step 1/10),

Qwen3.5-4B Q4_K_M + `\{\}\texttt{-no-mmproj-offload}`:

- \{\}\textbf{解析率 0/4}, 目标准确率 0/4, 平均延迟 534 s/帧;
- 失败原因: 单帧生成超时 (>600 s) 与 LM Studio 客户端连接抖动;
- 真值全部 \{\}\texttt{has\{\}_target=True}, 模型即使输出也未能在时限内返回可解析 JSON。
- 单独用 \{\}\texttt{max\{\}_tokens=256} 重跑一帧 step 2 的测试见

\{\}\texttt{experiment\{\}_vlm\{\}_one\{\}_frame\{\}_4b.json}(\{\}\textbf{78.7 s 后连接错误, 仍未解析成功})。

1.7.3 6.3 结论

- 本地 4B/9B/E4B 在 Intel 核显上\{\}\textbf{仅适合离线数据标注}; 在线逐步控制应继续使用

云端 mimo-v2.5 (约 9.5 s/帧) 或更强的本地 GPU。

- 视觉管线瓶颈在 \{\}\textbf{CPU mmproj 编码} (~30–90 s) 和 \{\}\textbf{低 tok/s 生成}, 二者叠加后

单帧常超 5 min; QLoRA 微调前应先固定 \{\}\texttt{-no-mmproj-offload} 与足够长的 HTTP 超时。

1.8 7. 训练准备 (P7, 等 ssh5090)

- \{\}\texttt{vlm-training-data-processed-runs/}: 22 游戏 3054 步 → \{\}\textbf{15,083 样本 / 7 任务}

(next_probe_action 2645 / probe_action_effect 2645 / field_grounding 2645 / information_gain 3054 / pulse_response 1435 / progression 2645 / failure_recovery 14), VLMColdStartDataset 加载验证通过, 14 单测。

- failure_recovery 仅 14 条 (成功 run 中卡死窗口少) ——建议用 verifiers 环境跑对抗性失败 run 补数据 (rule 变体/随机扰动注入)。

- \{\}\texttt{train\{\}_qwen35.py} (QLoRA 4bit NF4 + ZeRO-2) 就绪; gemma4-e4b 需 transformers 补丁复核。
- ssh5090 可访问后: \{\}\texttt{bash scripts/scp\{\}_to\{\}_ssh5090.sh} + 数据同步 + 开训。

1.9 8. verifiers 风格环境 (P6)

`\{\}\texttt{src/experiments/game\{\}\_env.py}`: score_trajectory 五轴 rubric (completion/progress/activity/consistency/composite), 可直接评分历史 run JSON, 也可 GameEnv.rollout 在线跑分。8 单测。为后续 RL 微调提供 env+rubric 抽象。

1.10 9. 给同学四个问题的直接回答

1. `\{\}\textbf{空间一致性}`: 版本化世界模型 + 失效检测 (stale/面板/能力翻转) + 分级恢复;

关键是「交互方式失效」要有一等公民的检测通道 (本游戏是 failCount 翻转, 其他游戏可能是 autoFishing 类能力标志)。

2. `\{\}\textbf{时间一致性}`: 三层时间戳对齐 + 阶段契约 (precondition/allowed_actions/success+settle+timeout) + 前缀 hash 防策略改写 + 失败分级。

3. `\{\}\textbf{策略优化}`: 引导式 prompt 的方向对, 但先把 `\{\}\textbf{输出契约}` (token 预算、末行 JSON、解析回退) 做扎实; 最优性评估用模拟器穷举做真值, 别让模型自己评。

4. `\{\}\textbf{效率}`: 默认 L0 状态探针 + 触发器升级; DEBUG 环形内存、触发窗口落盘; 回退分级 (行为/状态/策略)。

1.11 10. 后续建议 (优先级序)

1. 面板处理泛化: 把 LosePanel 个案抽象为「模态面板阻断」检测器 (任意 Panel 激活 + 输入无效化), 进阶阶段契约守卫。

2. ssh5090 QLoRA: 先 qwen3.5-4b (管线已验证), 数据用 cold-start + processed-runs 合并。

3. 用 verifiers 环境批量产 failure_recovery 数据 (对抗性失败注入)。

4. 9B/E4B 基准补齐; VL struct 准确率上去后替代 mimo 视觉降本。

5. 22 游戏 benchmark: 每游戏 profile + 3 seed × 2 配置 (机制开/关) 配对复跑。